

Contents

1. What is ThreatGraph	2
2. What We Build	2
3. Open-Source Tools We Use	2
3.1 Discovery and Scanning	2
3.2 Backend and Data	3
3.3 Infrastructure and Orchestration	3
3.4 Security Hardening Tools	3
4. How It Works	4
5. Architecture	5
6. Risk Scoring	6
7. Project Risks	7
8. Roadmap	7
9. Expected Outcomes	8
10. Compliance	9

1. What is ThreatGraph

ThreatGraph is a tool that finds every internet-facing asset your company has, checks which ones are vulnerable, and tells you exactly what to fix first. Think of it as a security scanner that never stops running. It discovers domains, IPs, open ports, web apps, cloud resources, and APIs that are exposed to the internet. Then it scores the risk and pushes the critical issues straight to your team so they can act fast.

Most companies do not know their full attack surface. Shadow IT, forgotten cloud instances, and misconfigured servers create blind spots that attackers exploit. ThreatGraph eliminates those blind spots by continuously discovering and monitoring everything that faces the internet.

What It Does	Why It Matters
Finds all internet-facing assets	You cannot protect what you cannot see
Scores risk by business impact	Fix what matters, not what is loudest
Auto-creates remediation tickets	Reduces time from discovery to fix
Monitors 24/7 for changes	Catches new exposures before attackers do

Table 1: ThreatGraph at a Glance

2. What We Build

We build five core modules. Each one handles a specific job, and they all connect through an event bus so data flows automatically from discovery to remediation without manual handoffs.

Module	What It Does	Key Feature
Discovery Engine	Finds assets using passive + active + cloud scans	Zero-footprint passive recon
Asset Inventory	Stores and deduplicates all found assets	Time-series history per asset
Risk Engine	Scores vulnerabilities by real-world impact	CVSS + EPSS + business context
Remediation Hub	Creates tickets, triggers auto-blocks	Jira/ServiceNow integration
Monitoring Loop	Watches for changes 24/7, alerts on new exposures	Real-time diff detection

Table 2: Five Core Modules

3. Open-Source Tools We Use

Every tool listed below is open-source. We picked them for three reasons: they are battle-tested in production, they have active communities, and they integrate well with each other. No vendor lock-in, no license costs.

3.1 Discovery and Scanning

Tool	What It Does	Why We Use It
Nuclei	Scans for 2000+ known CVEs using templates	Fast, community-updated templates, custom rules

Tool	What It Does	Why We Use It
Nmap	Port scanning and service detection	Industry standard, reliable version detection
Masscan	Scans entire IP ranges at wire speed	Fastest port scanner available, covers /8 ranges
ZMap	Internet-wide IPv4 scanning	Research-grade scanner, 1Gbps+ throughput
Subfinder	Discovers subdomains from passive sources	Passive only, zero footprint, 40+ data sources
HTTPX	Probes web servers, grabs titles and tech	Fast, filters live hosts from dead ones
Amass	Network mapping and asset discovery	OSINT + active, builds asset relationships

Table 3: Discovery and Scanning Tools

3.2 Backend and Data

Tool	What It Does	Why We Use It
FastAPI	Python API framework for core services	Async, auto-docs, type-safe, fast dev
Go (gRPC)	High-throughput scan worker services	10x faster than Python for network I/O
TimescaleDB	Time-series database for asset history	PostgreSQL extension, SQL, auto-partitions
Redis	Cache and session store	Sub-ms reads, pub/sub for real-time events
Kafka	Event streaming between services	Decouples services, handles bursts, replay
Elasticsearch	Full-text search on asset data	Instant search across millions of assets

Table 4: Backend and Data Tools

3.3 Infrastructure and Orchestration

Tool	What It Does	Why We Use It
Kubernetes	Runs scan workers, auto-scales pods	De-facto standard, handles scale
Temporal.io	Workflow orchestration for scan jobs	Built-in retry, timeout, state tracking
Kong	API gateway with rate limiting and auth	Plugin-based, extensible, open-source
ModSecurity	Web Application Firewall	OWASP Core Rule Set, blocks attacks

Table 5: Infrastructure Tools

3.4 Security Hardening Tools

These tools make ThreatGraph itself secure. Since we handle sensitive vulnerability data, the platform must be hardened against the same attacks it detects for others.

Tool	What It Secures	How
Falco	Runtime security for containers	Detects anomalous behavior in K8s pods
Kyverno	Kubernetes policy enforcement	Blocks privileged pods, enforces resource limits
Trivy	Container image scanning	Scans our own images for CVEs before deploy

Tool	What It Secures	How
Vault (HashiCorp)	Secrets management	API keys, DB passwords, cloud creds rotated auto
OpenSSL/Let's Encrypt	TLS everywhere	Auto-renewing certs, no plaintext traffic
Keycloak	Authentication and RBAC	OIDC/SAML, MFA, role-based access per workspace

Table 6: Security Hardening Tools

4. How It Works

The system runs in a continuous loop of five phases. Each phase feeds the next, and the last phase loops back to the first. This means the system never stops improving your security posture.

Phase	Action	Result
1. Discover	Scan internet for assets (passive + active + cloud)	Complete asset list with IPs, ports, services
2. Inventory	Deduplicate, fingerprint, link related assets	Clean, linked asset database
3. Assess	Match CVEs, score risk, find attack paths	Prioritized list of what to fix
4. Remediate	Create tickets, auto-block where possible	Fixes in progress, critical items blocked
5. Monitor	Watch for changes, re-scan on drift, tune rules	New exposures caught within hours

Table 7: The Five-Phase Loop

ThreatGraph Operational Workflow



Figure 1: Operational Workflow

5. Architecture

The system is built in layers. Data flows up from discovery sources at the bottom to the API gateway at the top. Each layer does one job and talks only to its neighbors. This keeps things simple and lets us scale each layer independently when load increases.

Layer	Component	Job
L7	Kong + ModSecurity	API gateway, auth, WAF, rate limiting
L6	Temporal.io	Schedule and coordinate scan jobs
L5	Elasticsearch + ClickHouse	Search assets, aggregate risk stats
L4	Kafka	Move events between services

Layer	Component	Job
L3	FastAPI + Go gRPC	Core business logic and APIs
L2	TimescaleDB + Redis	Store asset data, cache hot queries
L1	Kubernetes + Dask	Run distributed scan workers
L0	Nuclei, Nmap, Shodan, etc.	Gather data from the internet

Table 8: Architecture Layers

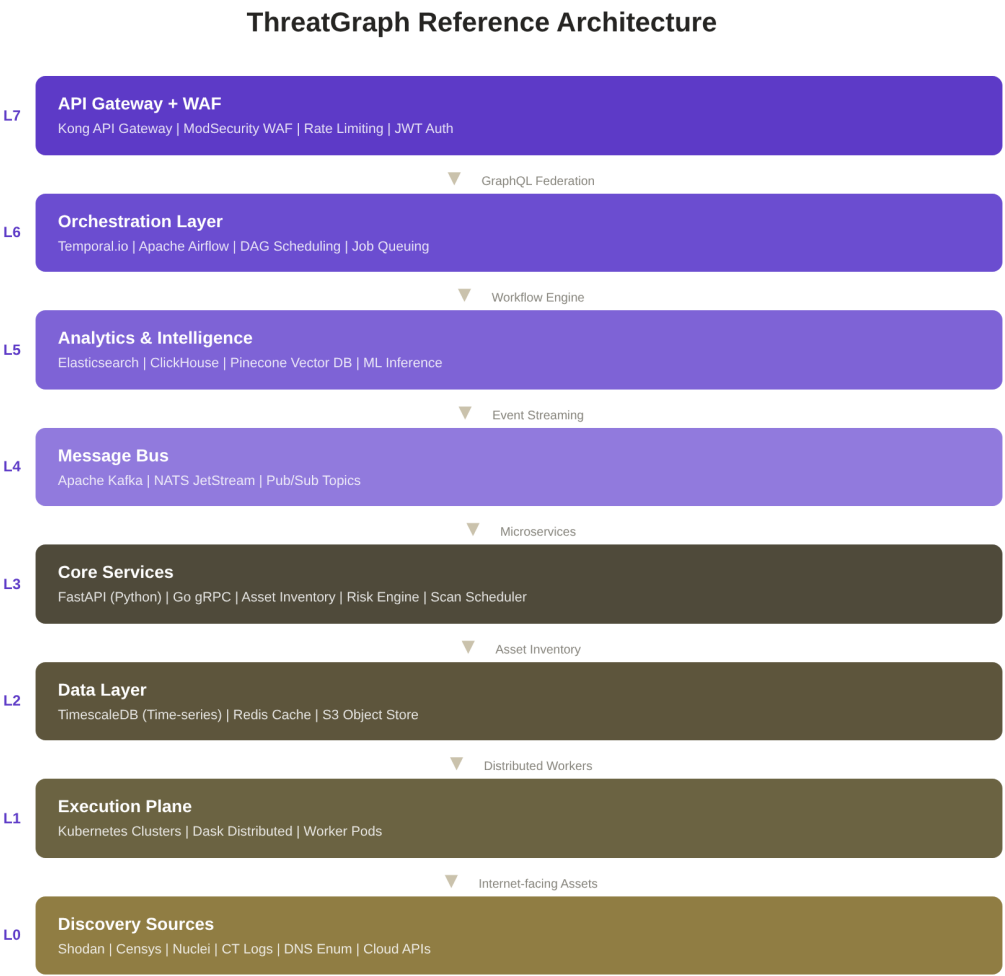


Figure 2: Layered Architecture

6. Risk Scoring

Not all vulnerabilities are equal. A critical CVE on a public-facing database is an emergency. The same CVE on an internal test server is a low priority. ThreatGraph scores risk using three inputs: technical severity, exploit likelihood, and business impact. The result is a single priority number that tells your team exactly what to fix first.

Input	Source	What It Tells You
Technical Severity	CVSS 4.0	How bad the vulnerability is in theory
Exploit Likelihood	EPSS	Probability it gets exploited in next 30 days
Business Impact	Asset criticality + data class	How much damage if this asset is compromised
Attack Path	Graph analysis	Can an attacker chain this to reach a bigger target
Blast Radius	Dependency mapping	How many other systems go down with this one

Table 9: Risk Scoring Inputs

7. Project Risks

These are the real risks to the project itself, not the vulnerabilities the platform finds. Each one has a specific mitigation already in place.

Risk	Chance	Impact	Fix
External APIs rate-limit our scans	High	Medium	Multi-provider fallback, cached results
Cloud API credentials expire	Medium	High	Auto health checks, graceful degradation
Too many false positives	Medium	High	ML feedback loop, analyst tuning
Kafka bottleneck at peak load	Low	High	Topic partitioning, backpressure
Cross-tenant data leak	Low	Critical	DB sharding per workspace, RBAC, encryption

Table 10: Project Risk Register

8. Roadmap

We build in five phases over 18 months. Each phase delivers working software. No big-bang release at the end.

Phase	When	We Build	Delivered
1. Foundation	M1-3	Discovery engine, asset inventory, basic dashboard	10K+ assets found, inventory live
2. Scanning	M4-6	K8s scan workers, Nuclei integration, cloud connectors	95% coverage, CVE scoring active
3. Intelligence	M7-9	Risk scoring, attack paths, SIEM integration	Risk ranking in < 1 hour
4. Automation	M10-12	Auto-remediation, ticketing, auto-block, monitoring	MTTR < 7 days, 20% risk drop/quarter
5. Scale	M13-18	ML anomaly detection, vector search, graph modeling	Full CTEM platform

Table 11: 18-Month Roadmap

ThreatGraph Implementation Roadmap

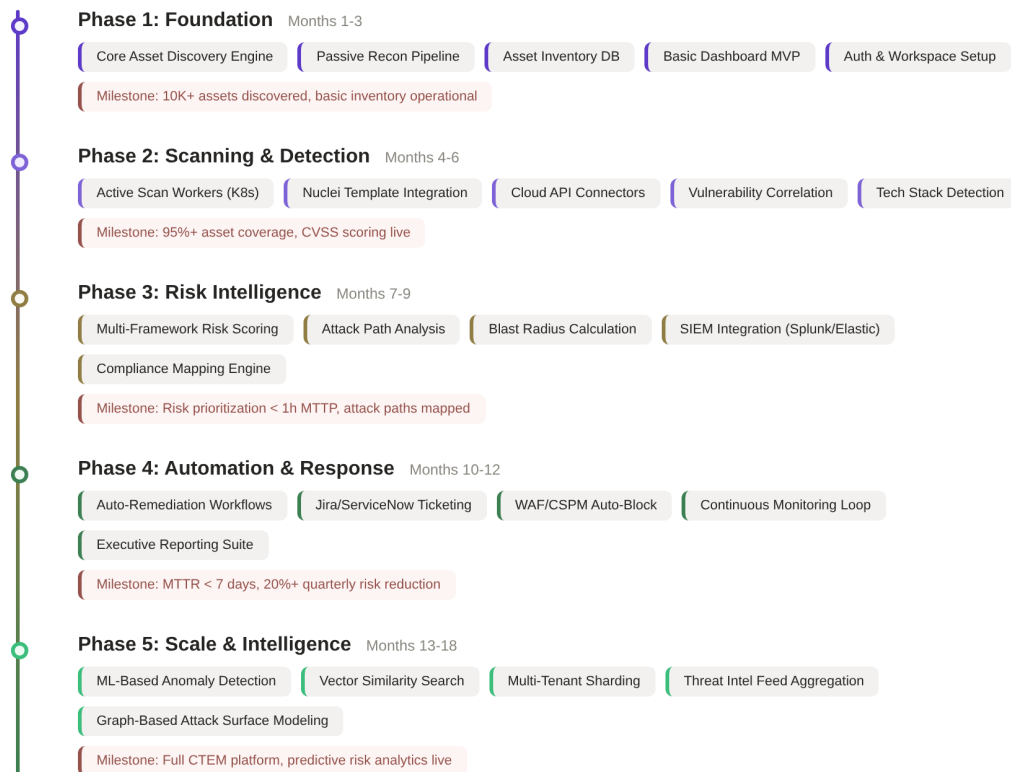


Figure 3: Implementation Roadmap

9. Expected Outcomes

Here is what changes after ThreatGraph is deployed. These are measurable targets, not vague goals.

What Changes	Before	After
Time to find a new exposure	2-4 weeks	< 24 hours
Risk prioritization	Manual triage	Automated scoring
Time to fix critical issues	30-90 days	< 7 days
Manual analyst work	40% of time	< 10% of time
Compliance evidence	Scramble once a year	Always available
Quarterly risk trend	Flat or rising	20%+ reduction

Table 12: Before vs After

10. Compliance

ThreatGraph automatically generates evidence for these frameworks. No more manual evidence collection during audits.

Framework	Controls Covered	What ThreatGraph Provides
NIST 800-53	RA-3, RA-5, CA-7	Continuous risk assessment logs
PCI-DSS	Req 6, 11.2, 11.3	Vuln scan reports, remediation evidence
GDPR	Art 32	Technical measures proof
SOC 2	CC6.1, CC6.6	Access controls, change tracking
MITRE ATT&CK;	TA0043	Reconnaissance detection data

Table 13: Compliance Coverage